

HAIA: Agentes para PyCatan

Shiyi Cheng

Codex

21 de abril de 2026

1. Objetivo y estado actual

El objetivo del trabajo es desarrollar un agente para Catan y evaluarlo de forma experimental, cubriendo:

- Parte 1: agente heurístico y benchmarks frente a random y agentes estándar.
- Parte 2: integración de LLM en dos decisiones del agente y análisis comparativo de prompts/modelos.

En esta entrega parcial se ha completado de forma sólida la Parte 1 y se ha dejado implementada la infraestructura de la Parte 2 (pipeline y agente LLM), aunque faltan experimentos finales con UPV y Bedrock para cerrar esa parte con datos definitivos.

2. Cambios implementados

2.1. Refactor de benchmarks

Se unificaron los benchmarks en una infraestructura común:

- `PyCatan/benchmark_common.py`
- wrappers: `benchmark_vs_random.py` y `benchmark_vs_agentes_estandar.py`

Mejoras introducidas:

- ejecución reproducible por semillas múltiples;
- misma estructura de métricas y CSV en ambos benchmarks;
- configuración por variables de entorno para agentes, seeds, partidas y workers;
- soporte robusto en este entorno (fallback a `ThreadPoolExecutor` cuando `ProcessPoolExecutor` falla por permisos).

Campos homogéneos de salida: `wins`, `winrate`, `avg_points`, `avg_rank`, `fallback_rate`, `provider`, `model`, `seed`.

2.2. Agente base incremental

Se implementó un nuevo tronco:

- `ShiyiBaseAgent`, que hereda de `RandomAgent`.

Este agente usa *feature flags* por decisión:

- `on_game_start`
- `on_build_phase`

- `on_commerce_phase`
- `on_moving_thief`

Si una mejora esta desactivada o falla en runtime, el comportamiento vuelve a random (fallback seguro).

2.3. Agente heuristico y agente LLM

- `ShiyiHeuristicAgent` ahora extiende el tronco incremental.
- `ShiyiLLMAgent` mantiene la logica LLM y ahora acepta configuracion por diccionario (facilita barridos de prompts/modelos desde benchmark).

3. Diseno de heurísticas (Parte 1)

El agente heuristico usa funciones de valor en:

- produccion esperada por nodo (ponderada por probabilidad de dado);
- diversidad de recursos;
- valor de expansion de carreteras;
- objetivo dinamico de construccion (ciudad, pueblo, carretera, carta);
- comercio con banca para cubrir escasez relevante.

La prioridad de construccion aplicada es:

ciudad > pueblo > carretera > carta

4. Protocolo experimental

4.1. Pipeline iterativo

Se implemento y ejecuto el pipeline:

- `iter0_random_baseline`
- `iter1_game_start`
- `iter2_build_phase`
- `iter3_commerce_trade`
- `iter4_thief_devcards`

Configuracion usada para el gate:

- seeds: 42, 31415, 2026
- 25 partidas por posicion (100 por seed)
- criterio: media de `winrate` $\geq 0,70$ vs random

Resultado consolidado: `PyCatan/benchmark_iterative_summary.csv`

5. Resultados

5.1. Ablation (vs random)

Iteracion	Media winrate	Desv. tipica	Min seed
iter0_random_baseline	0.2733	0.0047	0.2700
iter1_game_start	0.2800	0.0294	0.2500
iter2_build_phase	0.8433	0.0499	0.7900
iter3_commerce_trade	1.0000	0.0000	1.0000
iter4_thief_devcards	1.0000	0.0000	1.0000

Cuadro 1: Resumen del pipeline iterativo contra random.

Conclusiones directas:

- El mayor salto aparece en `iter2` al activar `on_build_phase`.
- `iter3` y `iter4` alcanzan 100 % en las seeds del gate.
- El requisito de 70 % se supera claramente desde `iter2`.

5.2. Validacion fuera de muestra (vs random)

Para comprobar que el 100 % no era un artefacto de las seeds del gate, se ejecuto una validacion extra con seeds no usadas en el diseno:

7, 99, 12345

Fuente: PyCatan/benchmark_vs_random_resultados.csv

Seed	Partidas	Victorias	Winrate
7	100	100	1.0000
99	100	100	1.0000
12345	100	100	1.0000

Cuadro 2: Iteracion 4 contra random, seeds fuera del conjunto original.

5.3. Comparacion contra agentes estandar

5.3.1. Iteracion 2 (gate minimo, primer estado competitivo)

Fuente: PyCatan/iter2_build_phase_benchmark_vs_estandar_resultados.csv

Seed	Partidas	Victorias	Winrate
42	500	203	0.4060
2026	500	180	0.3600
31415	500	209	0.4180

Cuadro 3: Iteracion 2 contra estandar.

5.3.2. Iteracion 4 (estado actual)

Fuente: PyCatan/benchmark_vs_estandar_resultados.csv

Seed	Partidas	Victorias	Winrate
42	500	460	0.9200
2026	500	451	0.9020
31415	500	454	0.9080

Cuadro 4: Iteracion 4 contra estandar.

6. Analisis: por que aparece winrate=1.0 en iteracion 4

No es un error de formato del CSV: en los experimentos mostrados se observan `wins=matches` y `avg_rank=1.0000`.

Interpretacion mas plausible con los datos actuales:

- el agente random del simulador es muy debil frente a una politica heuristica de construccion/comercio bien orientada;
- el salto principal viene de activar construccion y comercio (iter2 e iter3), no del modulo thief/development cards;
- el resultado se mantiene en seeds nuevas, por lo que no parece sobreajuste a las 3 semillas iniciales.

Importante: este 100 % se da **contra random**. Contra estandar, en el estado actual, el rendimiento ya no es perfecto pero sigue siendo alto (alrededor de 90 % en este entorno y esta configuracion).

7. Parte 2 (LLM): estado actual

Se ha implementado la infraestructura necesaria:

- decision LLM en `on_game_start` y `on_build_phase`;
- prompts: `compact_json`, `resource_focus`, `safe_legal`;
- script de evaluacion ligera: `PyCatan/evaluate_llm_phase2.py`;
- soporte de proveedores `upv` y `bedrock` en el motor.

Pendiente para cerrar la rubrica de Parte 2:

- ejecutar y documentar al menos 3 modelos distintos con datos finales;
- incluir comparativa de latencia/tokens/calidad/fallback entre prompts y proveedores.

8. Uso de herramientas de IA

Durante el desarrollo se usaron asistentes de IA para:

- refactorizar benchmarks y estandarizar metricas;
- implementar el tronco incremental con fallback seguro;
- preparar experimentacion iterativa y scripts de evaluacion LLM;
- redactar y revisar analisis de resultados.

Las decisiones de diseno y la validacion se apoyaron en ejecuciones reales y revision manual de resultados CSV.

9. Conclusiones

1. La Parte 1 esta conseguida con evidencia experimental fuerte: el objetivo de $\geq 70\%$ vs random se supera ampliamente.
2. El pipeline iterativo permite justificar que mejoras aportan mas rendimiento (especialmente build y commerce).
3. El resultado de 100% vs random en iteraciones avanzadas es consistente en varias seeds, incluidas seeds fuera del conjunto inicial.
4. Contra agentes estandar, el rendimiento en el estado actual es alto, lo que confirma que las mejoras no solo explotan debilidades triviales del random.
5. La Parte 2 esta implementada a nivel tecnico, pero requiere cerrar la experimentacion con modelos UPV/Bedrock para completar la practica al 100%.

10. Trabajo pendiente inmediato

- ejecutar matriz final de Parte 2 (≥ 3 modelos, ≥ 3 prompts, proveedores UPV/Bedrock);
- incorporar tabla final de latencia, tokens y fallback por configuracion;
- ajustar autores, revisar estilo final y limitar a maximo 10 paginas.